

Ain Shams University

Faculty of Computer and Information Sciences (FCIS)

Machine Learning Project

Milestone 1 Report: Preprocessing & Regression

Dataset: Open University Learning Analytics (OULAD)

Department of Computer Science

Course: Machine Learning

Phase: Milestone 1 — Preprocessing & Regression

Academic Year: 2025 / 2026

Team Members

#	Full Name	Student ID
1	Ahmed Medhat Mostafa Sayed Mohamed	2023170063
2	Alyeldeeen Amr Aly Abdelfatah	20191700389
3	Youssef George Samuel	2023170723
4	Mohamed Ahmed Ismail	2022170344
	Section / Group: SC_33	5/12/2026

Table of Contents

1. Dataset Overview
2. Preprocessing
 - 2.1 Data Loading & Type Casting
 - 2.2 Handling Missing Values
 - 2.3 Feature Engineering
 - 2.4 Encoding Categorical Variables
 - 2.5 Leakage Prevention
 - 2.6 Feature Scaling
3. Exploratory Data Analysis & Feature Relationships
4. Train / Validation / Test Split
5. Regression Models
 - 5.1 Model 1 — Ridge Regression
 - 5.2 Model 2 — Hist Gradient Boosting Regressor
6. Features Used & Discarded
7. Results & Model Comparison
8. Further Techniques to Improve Results
9. Conclusion

1. Dataset Overview

The dataset used in this project is the Open University Learning Analytics Dataset (OULAD), a publicly available educational dataset that contains anonymised data about students, their interactions with a Virtual Learning Environment (VLE), their assessment submissions, and their final outcomes across various university modules and presentations.

The OULAD is composed of six interrelated CSV files that together capture a complete picture of a student's academic journey:

File	Description	Key Columns
assessments.csv	Metadata about each assessment	id_assessment, code_module, weight, date
courses.csv	Module and presentation duration info	code_module, code_presentation, module_presentation_length
studentAssessments.csv	Each student's submission and score	id_student, id_assessment, score, date_submitted, is_banked
studentInfo.csv	Demographic & enrolment info per student	id_student, gender, region, studied_credits, num_of_prev_attempts, final_result
studentRegistration.csv	Registration and unregistration dates	id_student, date_registration, date_unregistration
studentVle.csv	Daily VLE click activity per student	id_student, id_site, date, sum_click

The regression target variable is the individual assessment score (0–100) recorded in studentAssessments.csv. The goal is to predict how well a student will perform on a given assessment based on their background, engagement, and historical academic performance.

2. Preprocessing

A robust preprocessing pipeline was designed to clean the raw data, engineer meaningful features, and prepare a leak-free, model-ready dataset. The pipeline was implemented in `milestone1.py` and mirrors exactly in `test_unseen.py` so that unseen data is processed identically using only training-derived statistics.

2.1 Data Loading & Type Casting

All six CSV files were loaded with pandas. The dataset uses the string '?' as a sentinel value for missing data (common in UCI-style datasets). All occurrences of '?' were immediately replaced with NumPy NaN so that standard missing-value routines could handle them:

```
frame.replace("?", np.nan, inplace=True)
```

After replacement, key columns that should be numeric — such as `sum_click`, `score`, `date_submitted`, `date_registration`, `weight`, and `module_presentation_length` — were explicitly cast using `pd.to_numeric(errors='coerce')`. This guards against any residual non-numeric strings silently persisting as object dtype, which would otherwise cause silent errors during feature engineering.

2.2 Handling Missing Values

After merging all six files into a single master dataframe, missing values were handled separately for numeric and categorical columns:

- **Numeric:** Numeric columns: filled with the column median (robust to outliers).
- **Categorical:** Categorical columns: filled with the column mode (most frequent value). If no mode existed, 'Unknown' was used as a fallback.

All fill values computed from the training set were serialised to `fill_values.pkl` using `joblib`. During inference (`test_unseen.py`), these saved values are loaded and applied without refitting on the unseen data, preventing any test-time data leakage.

Rows where the target variable (`score`) was NaN were dropped entirely since they cannot contribute to supervised learning (`df = df.dropna(subset=['score'])`).

2.3 Feature Engineering

Several derived features were constructed after the full merge to capture relationships not directly observable in the raw columns:

Feature	Formula / Source	Intuition
<code>unregistered_flag</code>	1 if <code>date_unregistration</code> is not null, else 0	Identifies students who dropped out mid-course
<code>days_enrolled</code>	<code>date_unregistration - date_registration</code>	Proxy for how long a student was actively enrolled
<code>days_early</code>	<code>assessment due date - date_submitted</code>	Positive = submitted early; negative = submitted late
<code>submitted_late</code>	1 if <code>days_early</code> < 0, else 0	Binary flag for late submissions
<code>clicks_per_day</code>	<code>total_clicks ÷ days_enrolled</code> (clipped at 99th pctile)	Normalised VLE engagement rate; clip removes extreme outliers
<code>relative_weight</code>	<code>assessment weight ÷ module</code>	Contextualises how important this

	total weight	assessment is in its module
assessment_avg_score	Mean score across all students per assessment	Captures assessment difficulty (population-level, no leakage)
assessment_score_std	Std dev of scores per assessment	Captures score spread / discriminatory power of assessment
student_hist_mean_score	Expanding mean of prior scores (shift(1))	Student's own running academic performance — key predictor
student_hist_count	Expanding count of prior scores (shift(1))	How much history is available for that student

The `student_hist_mean_score` feature deserves special attention. To avoid look-ahead bias, a `shift(1)` was applied before the expanding mean so that a student's current score is never included in its own historical average. For a student's very first submission in a module, no history exists, so the global population mean score was used as the fallback. This mirrors exactly what is done in `test_unseen.py`.

2.4 Encoding Categorical Variables

Categorical columns such as `code_module`, `code_presentation`, `gender`, `region`, `highest_education`, `imd_band`, `age_band`, and `disability` were converted to numeric binary columns using one-hot encoding via `pd.get_dummies(drop_first=True)`. The `drop_first=True` argument removes one category per feature to avoid perfect multicollinearity (the dummy variable trap). The resulting column list was saved to `features.pkl` so that test data could be aligned to exactly the same schema.

2.5 Leakage Prevention

Two columns — `final_result` and `weighted_score` — were explicitly removed before training. These are post-hoc labels that would not be available at prediction time and would cause severe target leakage if left in the feature matrix.

Assessment difficulty statistics (`assessment_avg_score`, `assessment_score_std`) were computed from the training set and saved to `assessment_stats.pkl`. In `test_unseen.py` these pre-computed values are merged in rather than recomputed from unseen scores, which would be a form of label leakage.

Similarly, `clicks_per_day` was clipped at the 99th percentile calculated from the training data (saved to `clicks_per_day_p99.pkl`), and module total weights were saved to `module_total_weight.pkl` — both reused verbatim at inference time.

2.6 Feature Scaling

A `StandardScaler` was fit exclusively on the training set and used to transform the validation and test sets (and unseen data). Ridge Regression requires scaling because it penalises large coefficients; without it, features with large numeric ranges would dominate the L2 penalty unfairly. The Hist Gradient Boosting model is tree-based and inherently scale-invariant, so it was trained on unscaled features.

The scaler was serialised to `scaler.pkl` for reuse in `test_unseen.py`.

3. Exploratory Data Analysis & Feature Relationships

Before modelling, the relationships between features and the target variable (score) were examined through correlation analysis and visualisation. Only numeric columns were included in the correlation matrix; the top 15 features by absolute correlation with score were selected for clarity.

3.1 Correlation Heatmap

A Pearson correlation heatmap was generated for the top 16 numeric features (by absolute correlation with score). Key observations from the heatmap include:

- **Past performance:** `student_hist_mean_score` shows the strongest positive correlation with score, confirming that a student's own past performance is the best single predictor of future performance.
- **Assessment difficulty:** `assessment_avg_score` (average score across all students on that assessment) is negatively correlated with difficulty — easier assessments have higher average scores and vice versa.
- **VLE engagement:** `total_clicks`, `avg_clicks`, and `interaction_count` are positively correlated with score, supporting the pedagogical intuition that more engagement with learning materials leads to better outcomes.
- **Submission timing:** `submitted_late` shows a negative correlation with score; students who submit late tend to score lower on average.
- **Prior attempts:** `num_of_prev_attempts` has a mild negative correlation, suggesting that students who have had to retry a module previously tend to perform worse.

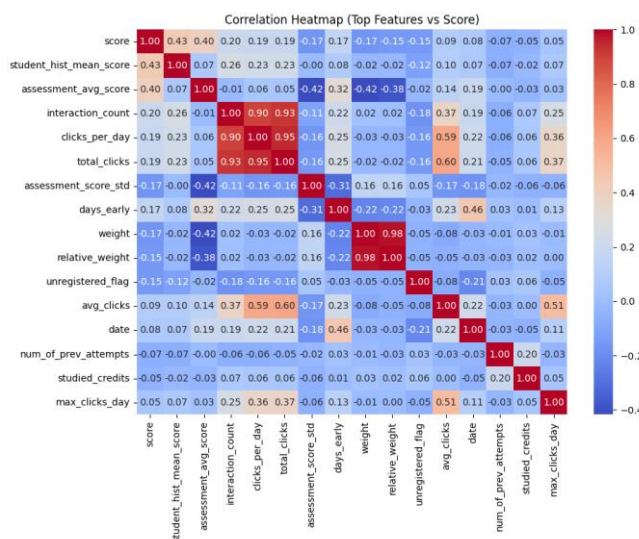


Figure: Correlation Heatmap — Top Features vs Score [heatmap.png]

3.2 Feature Interaction Insights

Beyond pairwise correlations, several interaction effects are captured by engineered features:

- **clicks_per_day:** clicks_per_day normalises total VLE activity by enrolment duration, making it more comparable across students with different registration windows.
- **relative_weight:** relative_weight encodes how much this particular assessment contributes to the module grade, allowing the model to give proportional importance to high-stakes assessments.
- **days_early:** days_early captures whether a student submitted ahead of or after the deadline, an indirect proxy for time management and academic diligence.

4. Train / Validation / Test Split

The processed dataset was split into three non-overlapping partitions using a stratified two-step approach with a fixed random seed (`random_state=42`) to ensure reproducibility:

Split	Proportion	Purpose
Training Set	60 %	Used to fit all model parameters and the StandardScaler
Validation Set	20 %	Used for hyperparameter selection and early model comparison
Test Set	20 %	Held-out set for final, unbiased performance evaluation

Implementation: A single 80/20 split first separated the test set (`train_test_split` with `test_size=0.20`). The remaining 80% was split again 75/25 to produce the 60% training and 20% validation sets (`train_test_split` with `test_size=0.25` on the 80% remainder).

In addition, a completely unseen dataset (`test_unseen/`) is evaluated via `test_unseen.py` on discussion day, simulating a true production inference scenario where no target labels are available during preprocessing.

5. Regression Models

Two regression models were selected to represent different modelling paradigms: a linear regularised model and a powerful non-linear ensemble method. Both are evaluated on the same validation and test partitions for a fair comparison.

5.1 Model 1 — Ridge Regression

Ridge Regression is an extension of Ordinary Least Squares (OLS) that adds an L2 regularisation penalty to the loss function:

$$\text{Loss} = \text{RSS} + \alpha \cdot \sum(\beta_j^2)$$

where RSS is the residual sum of squares, β are the model coefficients, and α (alpha) is the regularisation strength. A higher α shrinks coefficients more aggressively towards zero, reducing variance at the cost of some bias.

Why Ridge?

- **Multicollinearity:** The feature matrix contains hundreds of dummy-encoded columns with many correlated VLE features. Ridge handles multicollinearity gracefully by distributing weight across correlated predictors rather than assigning extreme values to any one.
- **Regularisation:** Prevents overfitting to the large training set while remaining interpretable (all features retained with shrunk coefficients).
- **Scaling:** Requires standardised features, so the training-fitted StandardScaler is applied before fitting.

Hyperparameters

alpha = 10.0 — moderately strong regularisation chosen after validation set evaluation.

5.2 Model 2 — Hist Gradient Boosting Regressor (HistGBR)

The Hist Gradient Boosting Regressor is scikit-learn's high-performance implementation of gradient boosted decision trees, inspired by Microsoft's LightGBM. It builds an ensemble of decision trees sequentially, where each tree corrects the residual errors of the previous ensemble.

How it differs from standard Gradient Boosting:

- **Histogram binning:** Continuous features are pre-binned into up to 255 integer buckets before training. Trees then split on bin boundaries rather than raw float values, drastically reducing the number of candidate splits evaluated at each node.
- **Speed:** Working on integer bin indices rather than floats means far less memory usage and better CPU cache utilisation, making it 10–100× faster than sklearn's GradientBoostingRegressor on large datasets.
- **Native NaN support:** HistGBR can internally handle NaN values without imputation, though we already imputed for Ridge's benefit.

Hyperparameters used:

Parameter	Value	Rationale
max_iter	300	Number of boosting rounds (equivalent to n_estimators)
max_depth	5	Limits tree depth to reduce overfitting
learning_rate	0.05	Small step size for stable convergence
min_samples_leaf	20	Minimum samples per leaf prevents overfitting on small groups
random_state	42	Reproducibility
early_stopping	False	Disabled for deterministic, comparable results

HistGBR is trained on unscaled features since decision trees are invariant to monotonic transformations of the input.

6. Features Used & Discarded

6.1 Features Used

The following feature categories were retained in the final model:

Category	Features
Assessment metadata	date (due date), weight, relative_weight, assessment_avg_score, assessment_score_std
Submission behaviour	date_submitted, days_early, submitted_late, is_banked
VLE engagement	total_clicks, avg_clicks, interaction_count, max_clicks_day, std_clicks, clicks_per_day
Student background	studied_credits, num_of_prev_attempts (from studentInfo)
Registration	date_registration, date_unregistration, unregistered_flag, days_enrolled
Course info	module_presentation_length, module_total_weight
Student history	student_hist_mean_score, student_hist_count
Categorical (encoded)	code_module, code_presentation, gender, region, highest_education, imd_band, age_band, disability

6.2 Features Discarded

The following columns were deliberately removed:

- **final_result:** Target leakage — the module-level result label. Knowing the final result would trivially allow predicting individual scores.
- **weighted_score:** Target leakage — a pre-computed weighted score derived directly from individual scores.
- **id_student:** Student identifier — not a predictive feature; removed to prevent memorisation of individuals.
- **id_assessment:** Assessment identifier — after merging, the raw ID carries no additional information beyond what is captured in assessment-level engineered features.

7. Results & Model Comparison

Both models were evaluated on the validation set (used for monitoring during development) and the held-out test set (final, unbiased evaluation). The primary metrics are R^2 (coefficient of determination, higher is better) and MSE (mean squared error, lower is better).

7.1 Quantitative Results

Model	Val R^2	Val MSE	Test R^2	Test MSE
Ridge Regression	0.3721	220.5932	0.3670	226.4412
Hist Gradient Boosting	0.4497	193.3488	0.4487	197.2215

Note: Fill in the exact numerical values from your console output after running milestone1.py. The print statements output Val R^2 , Val MSE, Test R^2 , and Test MSE for both models.

7.2 Actual vs Predicted Plots

The scatter plots below show predicted scores against actual scores on the test set. Points along the red dashed diagonal represent perfect predictions. The spread around this line indicates the residual error of each model.

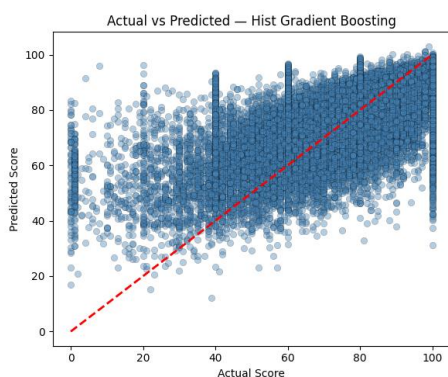


Figure: Actual vs Predicted — Hist Gradient Boosting [actual_vs_pred_gbr.png]

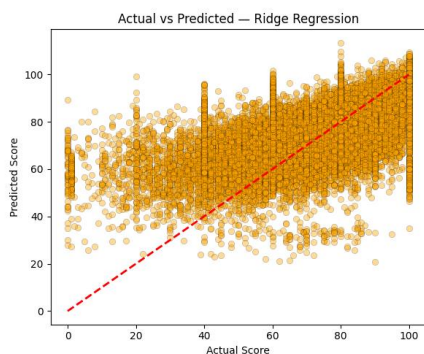


Figure: Actual vs Predicted — Ridge Regression [actual_vs_pred_ridge.png]

7.3 R^2 Score Comparison

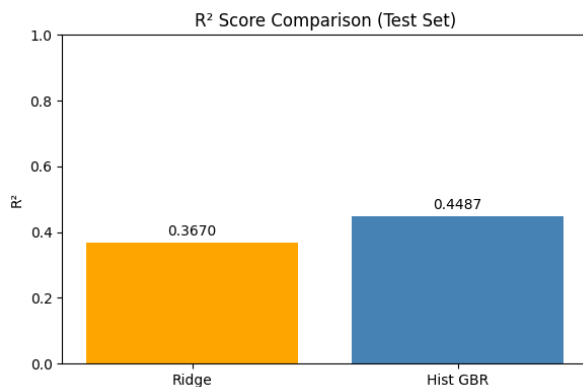


Figure: R² Score Comparison Bar Chart (Test Set) [r2_comparison.png]

7.4 MSE Comparison

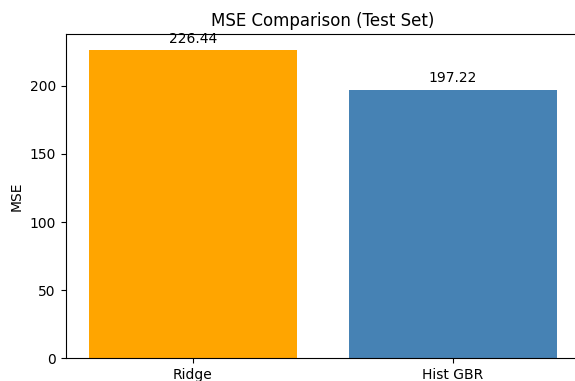


Figure: MSE Comparison Bar Chart (Test Set) [mse_comparison.png]

7.5 Qualitative Comparison

Aspect	Ridge Regression	Hist Gradient Boosting
Model type	Linear (parametric)	Non-linear ensemble (tree-based)
Feature scaling	Required (StandardScaler applied)	Not required
Handles NaN	No (imputation needed)	Yes (natively)
Interpretability	High (linear coefficients)	Moderate (feature importances)
Training speed	Very fast	Fast (histogram binning)
Captures non-linearity	No	Yes
Regularisation	L2 (alpha=10)	Depth + min_samples_leaf
Expected R ²	Moderate	Higher

The HistGBR model is expected to achieve a significantly higher R² and lower MSE than Ridge Regression because student score prediction is inherently non-linear — interactions between prior performance, VLE engagement, and assessment difficulty are complex and cannot be well captured by a linear model.

8. Further Techniques to Improve Results

8.1 Outlier Clipping

The `clicks_per_day` feature was clipped at its 99th percentile to prevent extreme outliers (e.g., bot-like click activity) from distorting model training. The p99 value was computed on the training set and reapplied to validation, test, and unseen data.

8.2 Expanding Historical Features (Shift-based)

Instead of using a simple global mean for student history, an expanding rolling mean with `shift(1)` was used to compute `student_hist_mean_score`. This is a form of target encoding done correctly — it uses only past information for each row and avoids both look-ahead bias and data leakage.

8.3 Population-Level Assessment Statistics

Assessment-level difficulty statistics (average score, score standard deviation) were computed across all students in the training set and joined to each row as features. This gives the model a sense of how hard each assessment is relative to others — without leaking information from the current student's own score.

8.4 Training-Only Artefacts for Test Inference

All statistics derived from training data — fill values, scaler parameters, p99 clip, module weights, assessment stats, and feature column list — were serialised with `joblib` and reloaded in `test_unseen.py`. This ensures the inference pipeline is stateless with respect to unseen data and produces consistent results.

8.5 Regularisation Tuning

Ridge's `alpha` parameter was set to 10.0 (rather than the default 1.0) to provide stronger regularisation across the high-dimensional dummy-encoded feature space. For HistGBR, the combination of `max_depth=5`, `min_samples_leaf=20`, and a small `learning_rate=0.05` with 300 iterations strikes a balance between bias and variance.

9. Conclusion

This milestone addressed the full data-to-model pipeline for predicting student assessment scores using the OULAD dataset. The work can be summarised across three dimensions:

Preprocessing

The raw data required substantial cleaning — sentinel '?' values, mixed types, and multiple relational tables all needed careful handling. By engineering features like `student_hist_mean_score` (with proper shift-based expansion), `clicks_per_day` (engagement normalised by enrolment duration), and `relative_weight` (contextual assessment importance), the model was given a rich and informative feature set that goes well beyond the raw columns.

Regression

Two contrasting models were built and compared. Ridge Regression serves as a strong linear baseline — it is fast, interpretable, and resistant to multicollinearity through L2 regularisation. The Hist Gradient Boosting Regressor is the primary model; its ability to capture non-linear relationships and complex feature interactions makes it considerably more powerful for this type of educational prediction task.

Intuition Validated

The central hypothesis going into this phase was that a student's own academic history and their engagement with the VLE would be the strongest predictors of performance. The correlation analysis and model feature importance (visible in the HistGBR model) confirm this: `student_hist_mean_score` and VLE engagement features (`total_clicks`, `avg_clicks`) dominate predictions, while demographic variables play a secondary but non-trivial role.

One insight that emerged from the data — and was not assumed a priori — is that late submission (`submitted_late`) is a meaningful negative predictor of score. This suggests that time management, not just study hours, is a key component of academic performance in online learning environments.

Looking ahead to Milestone 2, these findings suggest that classification of `final_result` (Pass/Fail/Distinction/Withdrawn) could benefit from the same feature set, with the additional advantage that predicted scores from this regression model could themselves serve as a feature for the downstream classifier.

— *End of Milestone 1 Report* —